Conceptual Architecture and Technical Workflow

Interval Multi-Objective DAG Workflow Offloading via
Graph-Attentive Evolutionary Optimization (GAT-IMOEA-DQN)

Contents

1 Conceptual Foundation and Paradigm Shift **2**

1.1 The Core Research Problem 2

1.2 Conceptual Solution: The GAT-IMOEA-DQN Framework 2

2 Mathematical System Model and Objective Formulation **3**

2.1 Three-Tier Continuum and DAG Application Model 3

2.2 Non-Deterministic Critical Path Timing Calculus 3

2.3 Tri-Objective Optimization Formulation 4

2.4 Multi-Aspect Uncertainty Dominance Relation (UDR) 4

3 End-to-End Algorithmic Workflow of GAT-IMOEA-DQN **5**

1 Conceptual Foundation and Paradigm Shift

1.1 The Core Research Problem

Modern distributed edge and cloud computing environments increasingly execute complex, multi-stage applications such as video analytics pipelines, autonomous vehicle coordination, and distributed microservice architectures. These applications cannot be decomposed into independent, isolated tasks; rather, they are structured as Directed Acyclic Graphs (DAGs) wherein subtasks exhibit strict precedence constraints and transfer intermediate data to downstream successors.

The foundational study by Jin et al. introduced an Interval Multi-Objective Evolutionary Algorithm guided by Deep Q-Networks (IMOEA-DQN) to optimize computation offloading under interval-valued communication bandwidth and computing frequencies. However, the base framework relies on an oversimplified assumption: every computing task is treated as an independent execution unit scheduled at a single decision epoch. Under this assumption:

- Inter-task dependencies are entirely ignored.
- Communication latency is modeled merely as the time to upload raw input data from a mobile device to an execution server.
- Total execution latency is computed as a trivial linear summation of individual task completion times.

In real-world workflow executions, offloading decisions cannot be evaluated via linear summation. When an upstream parent subtask and a downstream child subtask are offloaded to different physical nodes (for example, parent on an edge server and child in the centralized cloud), the intermediate output data must be transmitted across physical network links. The child subtask cannot begin execution until all of its parents have finished and all intermediate data transfers are complete. Consequently, the total execution time of the application is governed by the makespan along the non-deterministic critical path of the DAG, rather than an uncoordinated sum of task durations.

Furthermore, the original 7-dimensional state vector utilized by the DQN selector in Jin et al. captures only macro-level population dynamics (centroid movements, cluster counts, crowding distance variations, and overall interval metric shifts). It possesses zero structural awareness of the underlying application graph. When scheduling complex workflows, the evolutionary optimizer's ability to balance convergence, diversity, and uncertainty depends directly on topological characteristics—such as workflow branching width, critical path depth, and data-to-computation ratios.

1.2 Conceptual Solution: The GAT-IMOEA-DQN Framework

This research establishes an interval multi-objective workflow scheduling paradigm that bridges topological graph representation learning with uncertainty-aware evolutionary optimization. The proposed architecture introduces:

1. **Interval Critical Path Calculus:** Formalizes execution makespan, multi-tier energy dissipation, and monetary resource leasing under bounded interval arithmetic, accounting for inter-node intermediate data transfers and processor sharing.
2. **Two-String Chromosome Encoding with Topological Invariance:** A dual-chromosome structure combining an allocation vector with a priority ordering permutation, operated on by Precedence-Preserving Order-Based Crossover (POX) and topological repair mechanisms to guarantee deadlock-free execution.
3. **Graph Attention Network (GAT) Structural State Encoder:** A deep graph neural network that embeds the topological dependencies and interval weight bottlenecks of the DAG, producing an expressive structural embedding that conditions the reinforcement learning selection policy.

4. **Closed-Loop Uncertainty-Aware Environmental Selection:** A deep reinforcement learning agent that processes both the topological graph embedding and dynamic evolutionary population statistics to adaptively modulate the selection pressure across interval Pareto convergence, crowding diversity, and objective uncertainty.

2 Mathematical System Model and Objective Formulation

2.1 Three-Tier Continuum and DAG Application Model

The physical execution environment comprises a three-tier computing continuum:

- A centralized cloud data center C offering high computing capacity $\tilde{f}_C = [\underline{f}_C, \overline{f}_C]$ per task, accessed via a wired backhaul link with uncertain bandwidth $\tilde{W}_C = [\underline{W}_C, \overline{W}_C]$.
- A set of n distributed edge nodes $N = \{1, \ldots, n\}$, where each edge server $u \in N$ provides aggregate processing capacity $\tilde{f}_u = [\underline{f}_u, \overline{f}_u]$.
- A set of heterogeneous mobile terminal devices $M = \{1, \ldots, m\}$ with local processor clock frequencies $\tilde{f}_i = [\underline{f}_i, \overline{f}_i]$.

A submitted application is modeled as a Directed Acyclic Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$:

- $\mathcal{V} = \{v_1, \ldots, v_K\}$ represents K interdependent subtasks. Each subtask v_k has an interval computational requirement $\tilde{C}_k = [\underline{C}_k, \overline{C}_k]$ (in Gigacycles).
- $\mathcal{E} = \{(v_i, v_j) \mid v_i, v_j \in \mathcal{V}\}$ represents directed data-dependency edges. An edge (v_i, v_j) indicates that v_j cannot begin until parent v_i has completed and transferred intermediate data of size $\tilde{D}_{i,j} = [\underline{D}_{i,j}, \overline{D}_{i,j}]$ (in Kilobytes).
- Subtasks with $pred(v_k) = \emptyset$ are *entry tasks* $\mathcal{V}_{entry}$; subtasks with $succ(v_k) = \emptyset$ are *exit tasks* $\mathcal{V}_{exit}$.

The offloading decision for subtask v_k is $x_k \in \{0, 1, \ldots, n, n+1\}$, where $x_k = 0$ is local execution on the originating mobile terminal, $x_k = u \in \{1, \ldots, n\}$ is edge node u, and $x_k = n + 1$ is the centralized cloud.

2.2 Non-Deterministic Critical Path Timing Calculus

Under bounded interval algebra, addition follows $[\underline{a}, \overline{a}] + [\underline{b}, \overline{b}] = [\underline{a} + \underline{b}, \overline{a} + \overline{b}]$, scalar division follows $k/[\underline{a}, \overline{a}] = [k/\overline{a},\ k/\underline{a}]$ for $k > 0$, and interval maximization is $\max([\underline{a}, \overline{a}], [\underline{b}, \overline{b}]) = [\max(\underline{a}, \underline{b}), \max(\overline{a}, \overline{b})]$.

The effective computing capacity allocated to v_k on node x_k follows an equitable processor-sharing model:

$$\tilde{f}^{eff}_{x_k} = \begin{cases} \tilde{f}_i, & x_k = 0 \text{ (mobile device } i\text{)} \\ \dfrac{\tilde{f}_u}{|\{v_j \in \mathcal{V} \mid x_j = u\}|}, & x_k = u \in \{1, \ldots, n\} \text{ (edge server } u\text{)} \\ \tilde{f}_C, & x_k = n + 1 \text{ (cloud)} \end{cases}$$

The interval computation duration of v_k is

$$\tilde{t}_{comp}(v_k) = \frac{\tilde{C}_k}{\tilde{f}^{eff}_{x_k}}.$$

The interval transmission rate $\tilde{R}_{x_i,x_j}$ between nodes hosting dependent subtasks v_i and v_j is:

- If $x_i = x_j$: $\tilde{R}_{x_i,x_j} = \infty$, yielding zero transmission latency.
- If $x_i = 0$ and $x_j = u$: $\tilde{R}_{x_i,x_j} = \tilde{r}_{i,u} = \tilde{B} \log_2\left(1 + \dfrac{p_i g_{i,u}}{v_0}\right)$ (wireless uplink).

- If $x_i = u$ and $x_j = n+1$: $\tilde{R}_{x_i,x_j} = \tilde{W}_C$ (wired backhaul link).
- If $x_i = u_1$ and $x_j = u_2$ ($u_1 \neq u_2$): $\tilde{R}_{x_i,x_j} = \tilde{W}_{E2E}$ (edge-to-edge crosshaul link).

The interval ready time $\tilde{AT}(v_k)$ at which v_k receives all prerequisite input data is

$$\tilde{AT}(v_k) = \begin{cases} [0,0], & v_k \in \mathcal{V}_{entry} \\ \max\limits_{v_p \in pred(v_k)} \left(\tilde{FT}(v_p) + \mathbb{I}(x_p \neq x_k) \dfrac{\tilde{D}_{p,k}}{\tilde{R}_{x_p,x_k}} \right), & \text{otherwise} \end{cases}$$

where $\mathbb{I}(\cdot)$ is a binary indicator equal to 1 if the subtasks reside on different nodes and 0 if co-located.

The interval actual start and finish times on x_k must account for resource queue availability:

$$\tilde{ST}(v_k) = \max\big(\tilde{AT}(v_k),\ \tilde{avail}(x_k)\big), \tag{1}$$

$$\tilde{FT}(v_k) = \tilde{ST}(v_k) + \tilde{t}_{comp}(v_k), \tag{2}$$

where $\tilde{avail}(x_k)$ is the interval timestamp at which node x_k finishes executing all preceding subtasks assigned to its queue.

2.3 Tri-Objective Optimization Formulation

The framework optimizes three conflicting interval-valued objectives simultaneously.

1. Interval Workflow Makespan $\tilde{F}_1(X)$: total completion latency along the critical path to the terminal exit subtasks:

$$\min\ \tilde{F}_1(X) = \max_{v_{exit} \in \mathcal{V}_{exit}} \tilde{FT}(v_{exit}).$$

2. Comprehensive Three-Tier Energy Consumption $\tilde{F}_2(X)$: total energy dissipated across terminal devices, edge servers, and cloud:

$$\begin{aligned} \min\ \tilde{F}_2(X) &= \tilde{E}_{terminal}(X) + \tilde{E}_{edge}(X) + \tilde{E}_{cloud}(X), \\ \tilde{E}_{terminal}(X) &= \sum_{v_k:x_k=0} \tilde{C}_k \tilde{e}_i + \sum_{v_k:x_k\neq 0} p_i\, \tilde{t}^{upload}_{trans}(v_k) + \sum_{v_k:x_k\neq 0} p_i^{idle}\, \tilde{t}_{comp}(v_k), \\ \tilde{E}_{edge}(X) &= \sum_{u=1}^{n} \left(\kappa_E (\tilde{f}_u)^2 \sum_{v_k:x_k=u} \tilde{C}_k + P_u^{static} \cdot \max_{v_k:x_k=u} \tilde{FT}(v_k) \right). \end{aligned}$$

3. Monetary Resource Leasing Cost $\tilde{F}_3(X)$: aggregate financial expenditure for commercial edge and cloud infrastructure:

$$\min\ \tilde{F}_3(X) = \sum_{v_k:x_k\in\{1,\ldots,n\}} \eta^E\, \tilde{f}^{eff}_{x_k}\, \tilde{t}_{comp}(v_k) + \sum_{v_k:x_k=n+1} \eta^C\, \tilde{t}_{comp}(v_k),$$

where η^E is the unit billing price for edge execution (0.001 \$/(GHz·s)) and η^C is the cloud usage billing rate (0.01 \$/s).

2.4 Multi-Aspect Uncertainty Dominance Relation (UDR)

To prevent underestimating uncertainty or collapsing interval widths into arbitrary scalar averages, solutions are evaluated across two complementary geometric uncertainty metrics:

- **Cumulative Interval Length Sum:** $U_1(X) = \sum_{j=1}^{3} \big(\overline{F}_j(X) - \underline{F}_j(X)\big)$.
- **Hyper-Box Objective Volume Product:** $U_2(X) = \prod_{j=1}^{3} \big(\overline{F}_j(X) - \underline{F}_j(X)\big)$.

A candidate X_a uncertainty-dominates X_b ($X_a \prec_{UN} X_b$) if and only if

$$\forall p \in \{1,2\}:\ U_p(X_a) \le U_p(X_b) \quad \text{and} \quad \exists p' \in \{1,2\}:\ U_{p'}(X_a) < U_{p'}(X_b).$$

Evaluating both metrics avoids ranking anomalies: solutions with identical interval length sums can occupy vastly different hyper-volumes, and solutions running completely on local devices have zero cost intervals ($\overline{F}_3 - \underline{F}_3 = 0$), which would collapse U_2 to zero despite large uncertainty in latency and energy.

3 End-to-End Algorithmic Workflow of GAT-IMOEA-DQN

The operational workflow coordinates topological graph encoding, evolutionary recombination, and reinforcement learning-driven environmental selection.

Step 1: Chromosome Representation and Population Initialization

Because subtasks are subject to strict dependency constraints, standard one-dimensional integer arrays can produce invalid cyclic dependencies or scheduling deadlocks during crossover. To ensure topological validity, each individual is a two-string entity:

- **Allocation String** ($X \in \{0,1,\ldots,n+1\}^K$): the k-th gene specifies the execution target node for v_k.
- **Scheduling Priority String** ($O \in \Pi(K)$): a permutation $[o_1,\ldots,o_K]$ representing a valid topological ordering of $\mathcal{G}$. For every edge $(v_i, v_j) \in \mathcal{E}$, v_i must appear before v_j in O.

During initialization, a population P_0 of size $N = 105$ is generated. For each individual, allocation genes are sampled within communication-range constraints, while priority strings are generated via randomized topological sorting (Kahn's algorithm with randomized tie-breaking among in-degree-zero nodes).

Step 2: Graph Attention Network (GAT) Structural State Encoding

At generation t, before environmental selection, the workflow DAG $\mathcal{G}$ is processed by a 2-layer Graph Attention Network to extract structural bottleneck features.

1. Node feature construction. Each subtask v_k has input feature vector $h_k^{(0)} \in \mathbb{R}^5$:

$$h_k^{(0)} = \big[m(\tilde{C}_k),\ w(\tilde{C}_k),\ in_deg(v_k),\ out_deg(v_k),\ depth(v_k)\big],$$

where $m(\tilde{C}_k) = \frac{\underline{C}_k + \overline{C}_k}{2}$ is the workload midpoint, $w(\tilde{C}_k) = \overline{C}_k - \underline{C}_k$ is the interval uncertainty width, and $depth(v_k)$ is the topological depth from entry nodes.

2. Edge feature construction. Each edge (v_i, v_j) has feature vector $e_{i,j} \in \mathbb{R}^2$:

$$e_{i,j} = \big[m(\tilde{D}_{i,j}),\ w(\tilde{D}_{i,j})\big].$$

3. Multi-head graph attention propagation. For layer $l \in \{1,2\}$, the attention coefficient between connected tasks v_i and v_j across H_a heads is

$$\alpha_{i,j}^h = \frac{\exp\Big(\mathrm{LeakyReLU}\big(\mathbf{a}_h^\top [\mathbf{W}_h h_i^{(l-1)} \,\|\, \mathbf{W}_h h_j^{(l-1)} \,\|\, \mathbf{W}_e e_{i,j}]\big)\Big)}{\sum_{k \in \mathcal{N}(i)} \exp\Big(\mathrm{LeakyReLU}\big(\mathbf{a}_h^\top [\mathbf{W}_h h_i^{(l-1)} \,\|\, \mathbf{W}_h h_k^{(l-1)} \,\|\, \mathbf{W}_e e_{i,k}]\big)\Big)},$$

$$h_i^{(l)} = \sigma\left(\frac{1}{H_a}\sum_{h=1}^{H_a}\sum_{j\in\mathcal{N}(i)}\alpha_{i,j}^{h}\mathbf{W}_h h_j^{(l-1)}\right).$$

4. Global attention pooling. Final-layer node representations are pooled using a learned gating vector to form a 16-dimensional graph embedding $\mathbf{g}_t \in \mathbb{R}^{16}$:

$$\mathbf{g}_t = \sum_{k=1}^{K}\mathrm{softmax}\big(\mathbf{w}_g^\top h_k^{(2)}\big)\cdot h_k^{(2)}.$$

5. State fusion. The graph embedding is concatenated with the 7 macro-level population dynamics features from Jin et al.:

$$s_t = \big[\mathbf{g}_t \parallel c_1^t, c_2^t, d_1^t, d_2^t, u_1^t, u_2^t, \alpha_t\big] \in \mathbb{R}^{23},$$

where

- c_1^t: population centroid shift distance between generations $t-1$ and t;
- c_2^t: average translation distance of population cluster centroids;
- d_1^t: number of distinct individual clusters in decision space;
- d_2^t: relative change in average interval crowding distance;
- u_1^t: change in aggregate interval length sum across the population;
- u_2^t: change in aggregate interval hyper-box volume across the population;
- $\alpha_t = t/G_{max}$: normalized evolutionary search progress.

Step 3: Reinforcement Learning Action Selection

The deep Q-network takes the 23-dimensional state s_t and evaluates Q-values across $H = 28$ discrete selection actions generated via Das–Dennis simplex partitioning ($q = 6$ divisions over 3 performance dimensions):

$$\mathbf{a}_t = (\alpha\%, \beta\%, \gamma\%), \qquad \alpha\% + \beta\% + \gamma\% = 1.$$

To balance exploration and exploitation, actions are chosen by an ϵ-greedy schedule with power decay $\epsilon_t = 0.3^{t/100}$. If a random draw $u \le \epsilon_t$, an action is sampled uniformly; otherwise the agent exploits:

$$\mathbf{a}_t = \arg\max_{a\in\mathcal{A}} Q(s_t, a; \theta).$$

Step 4: Three-Tier Environmental Selection

The parent population P_t and offspring O_t are merged into $P_M = P_t \cup O_t$ of size $2N = 210$. Candidates are sorted independently across three criteria:

1. **Convergence Ranking ($Rank_C$):** ascending sort on interval dominance degree $R_{IN}(x) = \sum_{y\prec_{IN}x} S_{IN}(y)$, where $S_{IN}(y)$ is the count of individuals dominated by y under lower-limit possibility degree.
2. **Diversity Ranking ($Rank_D$):** descending sort on Interval Crowding Distance (ICD) against its $\sqrt{2N}$ nearest neighbors:

$$ICD(x_a, x_b) = \frac{\sum_{j=1}^{3}\big|m(\tilde{F}_j(x_a)) - m(\tilde{F}_j(x_b))\big|}{\mu(x_a, x_b) + V(x_a) + V(x_b) + 1},$$

where $\mu(x_a, x_b)$ is the objective hyper-box overlap volume and $V(x)$ is the individual hyper-box volume.

3. **Uncertainty Ranking ($Rank_U$):** ascending sort on uncertainty dominance degree R_{UN}, evaluated via the dual-metric relation (U_1 and U_2).

The next population P_{t+1} of size N is assembled by sequential deduplication:

- Select top $\alpha\% \cdot N$ individuals from $Rank_C$ into P'_C.
- Select top $\beta\% \cdot N$ from $Rank_D$ (excluding those in P'_C) into P'_D.
- Select top $\gamma\% \cdot N$ from $Rank_U$ (excluding those in $P'_C \cup P'_D$) into P'_U.
- Form $P_{t+1} = P'_C \cup P'_D \cup P'_U$.

Step 5: Reward Calculation and DQN Policy Update

The reward r_t measures the relative improvement in the penalty-based interval boundary intersection (PIBI) indicator between successive generations:

$$r_t = \frac{PIBI_{t+1} - PIBI_t}{PIBI_t}.$$

The transition tuple $e_t = (s_t, \mathbf{a}_t, r_t, s_{t+1})$ is appended to the Experience Replay Buffer (ERB). Once $|ERB| \geq ERB_{thres} = 250$, the DQN is updated every 50 generations by sampling mini-batches and minimizing the temporal-difference loss with Double DQN updates:

$$L(\theta) = \mathbb{E}\Big[\big(r_t + \gamma_{rl}\, Q\big(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-\big) - Q(s_t, \mathbf{a}_t; \theta)\big)^2\Big].$$

Step 6: Mating Selection and Dual-Adaptive Elite Recombination

Parents for the next generation are recombined using operators designed for the two-string chromosome:

1. **Parent partitioning via interval fronts:** P_{t+1} is sorted into interval non-dominated fronts $\mathcal{F}^{IN}$. Candidates in the top half form the elite mating pool $TopP_I$, restricted to mating among themselves to preserve converged scheduling patterns. Candidates in the bottom half $BottomP_I$ mate with the entire population to maintain exploratory diversity.
2. **Adaptive probability via uncertainty fronts:** Candidates are sorted into uncertainty non-dominated fronts $\mathcal{F}^{UN}$ using the UDR. Individuals in the top half receive a low crossover probability $p_c^{low} = 0.7$ to preserve stable, low-uncertainty assignments; individuals in the lower half receive $p_c^{high} = 0.9$ to encourage structural reconfiguration.
3. **Allocation string crossover:** The Dual-Adaptive Elite Crossover (DAEC) recombines allocation genes across the designated mating pools.

Appendix: Key Parameters

Parameter	Value
Population size N	105 (merged pool $2N = 210$)
GAT layers / node features / edge features	2 / 5 / 2
Graph embedding dimension	16
DQN state dimension	23 (16 graph + 7 population)
Discrete actions H (Das–Dennis, $q = 6$)	28
ϵ-greedy decay	$\epsilon_t = 0.3^{t/100}$
ERB threshold / update period	250 / every 50 generations
Crossover probabilities p_c^{low}/p_c^{high}	0.7 / 0.9
Edge / cloud billing	0.001 \$/(GHz·s) / 0.01 \$/s